

CANet-B4 vs the NWRD baseline: results, architecture, pipeline and findings

Last updated 2026-09-22.

For a summary of the baseline paper itself, with its figures and a check of its claims, see [paper_summary.md](#).

This document compares our method, **CANet with an EfficientNet-B4 encoder**, against the baseline set by the NWRD dataset paper, and against the other models this project has produced along the way. For each claim it gives the evidence, the reasoning, and whether the claim is ready to be made.

Status. Every model-quality number in this project so far was measured under a protocol that cannot support a comparison (§5, §6). The final numbers come from `notebooks/nwr_d_experiments.ipynb`, which re-trains every model under one protocol (§7). The cells marked ⏳ **pending** should be filled from `runs/comparison.md` once that run finishes. The efficiency figures in §3.3 are real measurements taken on 2026-09-22.

1. Summary

Question	Answer now	Confidence
Does CANet-B4 segment rust better than the paper's UNet + APF?	Numerically yes by a wide margin (F1 0.896 vs 0.557), but the two numbers come from different test sets and protocols	Not yet claimable. Needs the re-run (§7)
Does CANet-B4 beat DeepLabV3+ R50 (internship report)?	No evidence either way. Every existing CANet figure (0.78–0.81 IoU) is <i>below</i> the report's 0.83, and none of them are comparable	Not yet claimable
Is CANet-B4 more efficient than the paper's UNet?	Yes: 10.4× fewer FLOPs and 4.0× faster on CPU at 512×512, measured	Claimable (for inference cost)
Is CANet-B4 more efficient than DeepLabV3+ R50?	Yes: 2.0× fewer FLOPs, 1.2× faster on CPU	Claimable , with the caveat in §3.3 that this comes mostly from the encoder
Is the CAM + asymmetric decoder itself more efficient?	No. The same-encoder DeepLabV3+ B4 costs about the same (35.7 vs 36.9 GFLOPs)	Measured. CAM's case has to rest on accuracy

2. What is being compared

ID	Source	Model	Role here
P	Anwar et al., <i>The NWRD Dataset</i> , Sensors 23(15):6942, 2023 (<code>sensors-23-06942.pdf</code>)	UNet + adaptive patching with feedback (APF)	Baseline , the published state of the art on NWRD
R	Internship report (<code>Syed_Muhammad_Abdur_Rahman_Final_Presentation.pdf</code>)	"Custom DeepLabV3+", ResNet-50 encoder; also DCNet	Earlier in-house model

ID	Source	Model	Role here
S1	<code>notebooks/archive/DL_Project_sprint1.ipynb</code>	Unrecoverable (see F6)	Earlier in-house baseline attempt
S2	<code>notebooks/archive/DL_Project_sprint2.ipynb</code> , <code>backend/results/results.json</code> , <code>backend/models/best_model.pth</code>	CANet-B4	First CANet runs
Ours	<code>nwrdr/</code> pipeline, <code>notebooks/nwrdr_experiments.ipynb</code>	CANet-B4, plus <code>UNET</code> (paper architecture), DeepLabV3+ R50, optional DeepLabV3+ B4	The fair re-run that produces the final numbers

3. Architecture comparison

3.1 The three architectures

	Paper UNet (P)	DeepLabV3+ R50 (R)	CANet-B4 (ours)
Encoder	4 conv blocks, 64→1024 channels	ResNet-50	EfficientNet-B4
Pretraining	None (trained from scratch)	ImageNet	ImageNet
Deepest feature stride	1/16	1/16 (dilated)	1/32
Context module	None beyond depth	ASPP: parallel atrous convs + image pooling	CAM : serial dilated <i>global flow</i> ($d = 2, 4, 8$) plus parallel <i>context flows</i> at pool scales 2/4/8, concatenated, then channel-attention re-fusion
Decoder	Symmetric, a skip connection at every scale	Light: 1/4 skip plus bilinear upsampling	Asymmetric : fuses 1/32 context with 1/4 low-level features (32→48 ch), two 3×3 convs
Output	Full resolution	×4 bilinear	×4 bilinear, Dropout2d before the head
Input size in the paper / here	128×128 patches / 512×512	– / 512×512	– / 512×512

3.2 Why CANet should suit wheat rust (the hypothesis being tested)

Wheat stripe rust in NWRD appears as **small, arbitrarily shaped, scattered lesions** on multi-leaf field images with busy backgrounds, occlusions and varying light (paper §3.3). Rust is a small minority of pixels: 7.2% of validation pixels in `nwrdr-patched`.

1. **Context separates rust from look-alikes.** Yellowing leaves, soil, sun glare and dead tissue share rust's colour. Telling them apart needs to know what surrounds a pixel: a lesion sits on a leaf, in a stripe. CAM's serial dilated flow widens the receptive field step by step, and the pooled context flows add region-level summaries at three scales. A plain UNet only gets context through depth, and the paper's model saw 128px patches, so it could never see beyond 128px of the downsampled image. *Evidence it helps: pending.* This is exactly what CANet vs `UNET` in §7 tests.
2. **Pretrained features matter on a 100-image dataset.** NWRD has 100 field images (paper Table 1). A from-scratch UNet has to learn edges and textures from those alone. ImageNet pretraining brings generic features and usually matters most when data is scarce. This is a confound, not a flaw: it is part

of *why* CANet-B4 might win, and the same-encoder ablation (DeepLabV3+ B4) separates encoder from head.

3. **Attention re-fusion should suppress false alarms.** The channel-attention gate after pre-fusion can down-weight context channels that fire on background. This targets the paper's main error mode: its precision (0.506) is lower than its recall (0.624), so it over-predicts rust.
4. **The known weakness is fine boundaries.** The head predicts at 1/4 resolution and upsamples $\times 4$ bilinearly, and only one skip connection (1/4) carries detail. Thin or tiny lesions can be blurred. The paper's UNet has full-resolution skips and should do better on boundary pixels. Watch recall on small lesions in the qualitative figure.

3.3 Efficiency (measured)

Measured on 2026-09-22 with `nwr.d.benchmark` at a $1 \times 3 \times 512 \times 512$ input on an Intel Core (Family 6 Model 142), 4 threads, PyTorch 2.7.1, fp32 on CPU, median of 3 runs. FLOPs count multiply-adds as 2 (`torch.utils.flop_counter`).

Model	Params (M)	GFLOPs @512 ²	CPU latency, bs 1 (ms)	vs CANet-B4
canet_b4 (ours)	29.39	36.9	955	–
canet_b4_twopass (as first written)	29.39	52.5	1741	1.42× FLOPs, 1.82× slower
unet (paper architecture)	31.04	385.3	3791	10.4× FLOPs, 4.0× slower
deeplabv3plus_r50 (report)	26.68	73.1	1139	2.0× FLOPs, 1.19× slower
deeplabv3plus_b4 (same encoder)	18.62	35.7	1194	0.97× FLOPs, 1.25× slower

Reasoning and caveats:

- **UNet is expensive for a structural reason, not an implementation one.** It keeps 64–128 channels at full and half resolution, and at 512² those layers dominate. The network is fully convolutional, so cost per pixel does not depend on patch size: the paper's 128px patches cost the same *per image area*. The paper does cut cost by downsampling images first (Lanczos, §3.4.1), which is a resolution trade we do not make.
 - **CANet vs DeepLabV3+ R50: most of the saving is the encoder.** Against the same-encoder DeepLabV3+ B4, CANet has the same FLOPs and 10.8M more parameters (the CAM's 256–512-channel 3×3 convs). CANet is still 20% faster on CPU than DLv3+ B4, likely because more of its compute runs at 1/32 resolution, where memory traffic is lower. That is a plausible explanation, not a measured one. **Conclusion: the efficiency claim for CANet as a whole is solid, but "CAM is cheap" is not a claim we can make.**
 - **The two-pass fix (F4) is a correction, not an architectural gain.** The original code ran the encoder twice. Removing that saves 30% of FLOPs and 45% of CPU time with bit-identical outputs. Report the fixed number; mention the fix only as an engineering note.
 - **GPU numbers are pending.** The Kaggle run measures GPU latency (fp32/fp16), throughput and peak memory on the same GPU for every model. Training time per epoch also comes from that run. The paper reports 4,791 min for UNet + APF on $2 \times V100$ (Table 3). It isn't comparable to our per-run training time: different hardware, 500 vs 30 epochs, and a different patch pipeline.
-

4. Pipeline differences, each with its effect on comparability

Aspect	Paper (P)	Ours	Why we differ	Effect on the comparison
Source data	100 full-resolution images (4608×3456 / 6016×4000), downsampled with Lanczos	abdur548/nwr-d-patched : pre-cut patches served at 512×512	The patched dataset is what this project was built on, and it fits a free Kaggle GPU	Different test unit. Main reason published numbers can't be compared directly
Split	Random 80/10/10 <i>by image</i>	Train/val/test folders of the patched dataset; how they were cut is unknown	Inherited	If patches of one photo sit in several splits, our scores are inflated (F1). The notebook now prints the dataset's README and filenames to check
Patch size / context	128×128, stride 32 (overlapping)	512×512	Bigger patches give CAM room to use context	Bigger receptive context per decision; favours context models by design
Patch selection	APF: rust patches ($\geq 1\%$ rust) plus, every 5 epochs, patches where the model produced false positives	All rust-positive patches plus a fixed 20% random sample of background-only patches (fixed data seed)	Simpler, deterministic, and identical for every model, so selection can't favour one model	APF is itself a contribution of the paper. Re-training <code>unet</code> under <i>our</i> sampling tests the architecture, not APF (§8)
Background in test	Table 3: all patches; Table 4: background-only removed (Li et al. protocol)	Every test patch; also rust-positive patches only ($\approx$ Table 4)	The paper argues removing background hides false positives (§5), and we agree, so the full test set is primary	Both views are reported so each paper table has a counterpart
Augmentation	Horizontal and vertical flips	Flips, 90° rotations, brightness/contrast, elastic	Standard for small segmentation datasets; applied equally to all models	Fair within our run; one more reason not to compare with the paper directly
Loss	Focal loss (best of focal / Dice)	0.7-Dice + 0.3-BCE with <code>pos_weight</code> from the training masks	Handles imbalance at both region and pixel level; the loss the CANet runs used	Same loss for all our models, so fair within our run
Optimiser / schedule	RMSprop, lr 1e-6, ExponentialLR, 500 epochs, batch 64	AdamW lr 5e-5, wd 1e-4, cosine warm restarts, 30 epochs, batch 8, fp16, clip 1.0; pretrained encoders frozen for 3 epochs	500 epochs is infeasible on free GPUs; the schedule is the one CANet was developed with	Threat to validity: these settings were tuned for fine-tuning pretrained models and may under-train the from-scratch <code>unet</code> (§8)
Threshold	Not stated (presumably 0.5)	Chosen on val (IoU-maximising, 0.05–0.95), frozen, applied once to test; 0.5 also reported	Tuning on the reported split inflates scores (F2)	The 0.5 column needs no tuning at all
Metrics	Rust-class P / R / F1 (+ IoU in Table 4), apparently averaged per image (F8)	Pooled over all test pixels: IoU, Dice (= F1), P, R, specificity, accuracy	Pooling is the standard definition and stays stable when many patches have no rust (F3)	Per-image vs pooled averaging changes numbers by itself

Aspect	Paper (P)	Ours	Why we differ	Effect on the comparison
Model selection	Not stated	Best validation IoU at 0.5	Uses val only, so test stays untouched	–
Repeats / statistics	One run	3 seeds; paired image-level bootstrap plus a seed-level paired t-test	One run can't separate a real gap from seed noise	Our result will carry confidence intervals
Hardware	2× Tesla V100 32 GB	One Kaggle T4 (or Colab GPU)	Availability	Efficiency is compared only within one device

5. Results comparison

5.1 Every number that exists today

#	Source	Model	Split, threshold	P	R	F1/Dice	IoU	Usable for the claim?
1	P, Table 3	UNet + APF (100 img, stride 32)	test, –	0.506	0.624	0.557	–	Reference only (different test unit)
2	P, Table 4	UNet + APF (stride 128, bg removed)	test, –	0.593	0.552	0.564	0.438	Reference only; internally inconsistent (F8)
3	P, Table 4	Octave-UNet + APF	test, –	0.580	0.497	0.529	0.316	Reference only
4	R, p. 7	DeepLabV3+ R50	val, 0.70 (tuned on val)	0.9174	0.8975	0.9074	0.8304	No: tuned on the reported split, protocol differs
5	R, p. 10	DeepLabV3+ R50 on "NWRD"	?	0.8728	0.7931	0.6483	–	No: F1 inconsistent with P/R (F7)
6	R, p. 10	DCNet on "NWRD"	?	0.7523	0.8231	0.5702	–	No: same inconsistency
7	S1, cell 16	Unknown (definition lost)	val, sweep capped at 0.70	–	–	–	0.7624	No: model unrecoverable (F6)
8	S2, cell 18 output	CANet-B4, 15 epochs	val, 0.75 (tuned on val)	0.9207	0.8726	0.8960	0.8116	No: tuned on the reported split
9	results.json	CANet-B4 (another run)	val, 0.79 (tuned on val, grid edge)	0.8649	0.8913	0.8779	0.7823	No: tuned on the reported split; doesn't match #8 (F5)

Reading the table. Row 8 vs row 1 is where "+0.34 F1 over the paper" comes from. Row 4 vs row 8 is where "CANet is worse than our own DeepLabV3+" comes from. Neither follows, for the reasons in the last column. The internship report itself shows how much protocol alone moves the numbers: the same DeepLabV3+ scores F1 **0.90 on "NWRDF"** and **0.65 on "NWRD"** (rows 4 vs 5), a gap almost as big as the whole CANet-vs-paper gap.

5.2 Final comparison (fill from runs/comparison.md)

Test split, threshold chosen on val, mean ± std over 3 seeds.

Model	IoU	Dice / F1	Precision	Recall	IoU @0.5	IoU, rust patches only
CANet-B4 (ours)	🕒	🕒	🕒	🕒	🕒	🕒
UNet (paper architecture)	🕒	🕒	🕒	🕒	🕒	🕒
UNet, tuned lr (unet_tuned)	🕒	🕒	🕒	🕒	🕒	🕒

Model	IoU	Dice / F1	Precision	Recall	IoU @0.5	IoU, rust patches only
DeepLabV3+ R50	🕒	🕒	🕒	🕒	🕒	🕒

CANet-B4 minus...	ΔIoU [95% CI]	p (bootstrap)	ΔDice [95% CI]	seed t-test p
UNet (paper), the stronger of <code>unet</code> / <code>unet_tuned</code> by val IoU	🕒	🕒	🕒	🕒
DeepLabV3+ R50	🕒	🕒	🕒	🕒

Decision rule, fixed before seeing the results so the conclusion can't be fitted to them: - *CANet improves on baseline X* if the mean test IoU is higher **and** the bootstrap 95% CI for ΔIoU excludes 0 **and** CANet wins on at least 2 of 3 seeds. - *Comparable* if the CI includes 0. Then the efficiency advantage (§3.3) is the contribution. - *Worse* if the CI lies entirely below 0. Report it as such. - The paper's published numbers are context, never the test of the claim.

6. Findings

Each finding lists the evidence, the reasoning, and what was done about it.

F1. The patched dataset may leak between splits. (*open, highest impact*) Evidence: `nwr-d-patched` has 8,160 train, 480 val, and some number of test patches cut from 100 source images. How the patches were split isn't documented in this repo. Reasoning: neighbouring patches of one photo share leaves, lighting and lesions. If they appear in both train and test, the test measures memorisation, which would explain in-house F1 around 0.90 against 0.56 in the paper. Action: the notebook now prints the dataset's `README.md`, `processing_stats.json` and sample filenames per split. If splits aren't image-disjoint, rebuild them by source image before trusting any number.

F2. Thresholds were tuned on the split being reported. (*fixed*) Evidence: S1 cell 16 and S2 cell 17 sweep the threshold on val, then report val metrics at the best value. S2 even picked 0.79, the edge of its grid. Reasoning: choosing the best of about 28 thresholds on the evaluation set is a small but real optimistic bias, and a grid-edge optimum means the true optimum wasn't searched. Action: the threshold is now chosen on val over 0.05–0.95, applied once to test, and flagged if it lands on the grid edge. Threshold-free results at 0.5 are reported too.

F3. The per-epoch IoU was computed wrongly. (*fixed*) Evidence: training curves peak near 0.58 while pooled IoU on the same model is about 0.81 (`results.json` history vs final metrics). Reasoning: the notebooks averaged IoU over batches, and a batch with no rust that is predicted perfectly scores $0/(0+\epsilon) = 0$ instead of 1. That dragged the curve down and **made the best checkpoint selection noisy**, since it partly rewarded predicting rust in background batches. Action: all metrics are now pooled over the whole split; selection uses pooled val IoU at 0.5. Unit-tested against direct counting.

F4. CANet ran its encoder twice per forward pass. (*fixed*) Evidence: `extract_features(x)` and `extract_endpoints(x)` were both called. In `efficientnet_pytorch` 0.7.1, `extract_endpoints(x)` [`'reduction_6'`] is the `extract_features` output (verified to be bit-identical). Reasoning: this is pure waste at inference. In training it also advanced batch-norm statistics twice per step and drew two different drop-connect masks, a small change to training dynamics. Action: one pass now. The existing checkpoint loads unchanged and gives identical outputs, with 30% fewer FLOPs and 1.8× faster CPU inference.

F5. The published CANet numbers come from different runs. (*resolved by the re-run*) Evidence: S2's saved output reports threshold 0.75 / IoU 0.8116 (15 epochs). `results.json` says 0.79 / 0.7823. `best_model.pth` was saved at **epoch 24**, so it comes from the 30-epoch configuration in the uncommitted notebook edits.

Reasoning: the README, app and dashboard showed numbers that don't belong to the shipped checkpoint. No single CANet figure could be defended. Action: the re-run produces one CANet result per seed, and `export/` pairs the served checkpoint with its own metrics.

F6. The Sprint 1 baseline can't be reproduced. (*superseded*) Evidence: its model-definition cell is missing (cell 11 is a stub ending in `...`). The title says DeepLabV3+ ResNet-50 while the config says `efficientnet-b4`, and its final-evaluation cell has an indentation error and never ran. Action: baselines are now defined in code (`nwrdr/models.py`) and re-trained.

F7. The internship report's "NWRD" F1 values don't match their own precision and recall. (*open, affects row 5–6*) Evidence: DeepLabV3+ "NWRD" P 0.8728, R 0.7931 imply $F1 = 2PR/(P+R) = \mathbf{0.831}$, but the report says 0.6483. DCNet "NWRD" P 0.7523, R 0.8231 imply **0.786**, not 0.5702. The NWRDF rows *are* consistent (for example $0.9207 / 0.8832 \rightarrow 0.9016$). Reasoning: the F1 was probably averaged per image, where images without rust score 0 and drag the mean down, while P/R were pooled. Mixing the two makes the rows impossible to compare. Action: our pipeline reports pooled values only and says so.

F8. The paper's Table 4 is also internally inconsistent. (*context*) Evidence: P 0.593, R 0.552 imply F1 **0.572** (reported 0.564). F1 0.564 implies $IoU = F1/(2-F1) = \mathbf{0.393}$ (reported 0.438). Table 3 row 6 is consistent ($0.506 / 0.624 \rightarrow 0.559$). Reasoning: this suggests per-image averaging too. The paper's numbers are approximate reference points, not exact targets.

F9. The report's significance tests don't support what they claim. (*context for the write-up*) Evidence: report pp. 14–16 run two-proportion z-tests on "accuracy" at "sample sizes" 100–5,000, with a "Notebook accuracy" of about 0.90. Those values match the model's **F1/Dice** (0.9074), not its pixel accuracy (0.978, p. 7). Reasoning: F1 is not a proportion of independent trials, so a proportion z-test doesn't apply. If the "samples" are pixels, they're heavily correlated within an image, and treating them as independent overstates significance. Significance also rises with the chosen sample size by construction, which is what the table shows. Action: our pipeline resamples **whole test images** (paired bootstrap), which respects the correlation, and adds a t-test across training seeds for run-to-run variance.

F10. The two in-house runs differed in more than architecture. (*fixed*) Evidence: S1 kept 20% of background patches (`pos_weight` 3.65); S2 kept 10% (2.96). Their threshold grids also differed (0.30–0.70 vs 0.25–0.79). Action: one config for every model: the same background subset (fixed data seed), loss, schedule and threshold grid.

F11. Credentials were committed. (*fixed in files; rotation pending with the owner*) Evidence: Kaggle and W&B API keys were hard-coded in both sprint notebooks and are in the git history on GitHub. Action: removed from the files. The keys must be rotated, because removing them doesn't undo publication.

7. How the final comparison is made, and why

Decision	Reasoning
Also train <code>unet_tuned</code> (lr 1e-3); keep the better UNet by val IoU	Shared settings were tuned for pretrained models. Giving the scratch UNet its own lr removes the objection that the baseline was under-trained; picking by val, not test, keeps the choice unbiased
Re-train the paper's UNet under our protocol	The paper's numbers are on a different test unit (\$4). Re-training its architecture beside CANet, with identical data, loss, schedule and evaluation, isolates the architecture. That is the only like-for-like test of "ours vs the baseline"

Decision	Reasoning
Keep DeepLabV3+ R50	It's the model the internship report presented as the best in-house result, so CANet has to be shown against it too
Optional DeepLabV3+ B4 ablation	Same encoder as CANet. The gap between them is the effect of CAM + the asymmetric decoder alone (§3.2 point 2)
One fixed background subset for every run	Sampling then can't favour any model; seeds vary only initialisation, shuffle order and augmentation
Threshold on val, report on test	Removes the optimistic bias of F2. Reporting at 0.5 too shows that conclusions don't depend on tuning
Pooled pixel metrics	Standard definition; robust to patches without rust (F3, F7)
3 seeds + image-level paired bootstrap + seed t-test	The bootstrap covers test-set sampling uncertainty; the t-test covers training randomness. Three seeds is the minimum that shows variance within a free-GPU budget
Efficiency on one device, random weights	Weights don't change cost; one device removes hardware differences. The two-pass variant is kept to document the F4 fix
Winner exported by val IoU, not test	The app's model is chosen without looking at test, so the published test number stays unbiased

8. Threats to validity

1. **Split leakage (F1)** could inflate *all* our models equally. That leaves the ranking meaningful but the absolute numbers optimistic. Check it before quoting absolute values against the paper.
2. **Shared hyperparameters may disadvantage the from-scratch UNet.** An lr of 5e-5 over 30 epochs suits fine-tuning pretrained encoders; a scratch UNet usually wants a higher lr. If CANet beat only that UNet, part of the gap could be under-training. **Mitigated:** the notebook flag `TUNE_UNET_LR = True` (the default) adds `UNET_TUNED`, the same UNet at lr 1e-3. The report takes whichever UNet run is better on *validation* IoU as the paper baseline, so CANet is judged against the stronger one. It costs about 2–3 GPU-hours per seed.
3. **APF isn't reproduced.** Our `UNET` tests the paper's *architecture*, not its *patch sampling*. A claim of "beats the paper's full method" would need APF implemented, or the result stated as "beats the paper's architecture under a common protocol".
4. **The paper's own test isn't reproduced.** Full images rebuilt from patches on the paper's split would need the original NWRD images and patch coordinates, which `nwr-d-patched` doesn't carry.
5. **The effect of pretraining is folded into "architecture".** CANet-B4 and DeepLabV3+ are pretrained; the paper's UNet isn't. That's a legitimate design advantage, but say so; the B4 ablation separates encoder from head.
6. **Resolution.** Every model sees 512² patches here; the paper downsampled full images first. Context-hungry models (CANet, DeepLabV3+) benefit more from larger patches than UNet does.

9. How to state the result once the run finishes

- **Efficiency, supportable now:** "At 512×512, CANet-B4 needs 36.9 GFLOPs, 10.4× fewer than the UNet used by the NWRD paper (385.3) and 2.0× fewer than DeepLabV3+ R50 (73.1), and runs 4.0× and 1.2× faster respectively on CPU." Add the GPU figures from the Kaggle run.
- **Accuracy, conditional on §5.2:** "Trained and evaluated under an identical protocol, CANet-B4 improves test IoU over the NWRD paper's UNet by Δ [95% CI a, b] (3 seeds)." Use this wording only if the decision rule in §5.2 is met.

- **Against the published paper:** quote the paper's 0.557 F1 as context and name the protocol differences (§4). Never put 0.557 and our number in one column as if measured the same way.

Appendix A: where each number comes from

Number	Source
Paper P/R/F1/IoU, training time, hyperparameters	sensors-23-06942.pdf : Tables 3–4, §3.4, §4.2
Report DeepLabV3+ / DCNet	Syed_Muhammad_Abdur_Rahman_Final_Presentation.pdf : pp. 6–7, 10, 14–16
S1 threshold sweep, pos_weight 3.65	notebooks/archive/DL_Project_sprint1.ipynb : cells 10, 16 (saved outputs)
S2 metrics 0.8116, pos_weight 2.96	notebooks/archive/DL_Project_sprint2.ipynb : cells 10, 17–18 (saved outputs, as committed at 19b5479)
0.7823 and training history	backend/results/results.json
Checkpoint epoch 24	backend/models/best_model.pth metadata
Efficiency table	nwr.d.benchmark.count_flops / time_forward , run locally on 2026-09-22
Final results	runs/comparison.md , runs/comparison.json , runs/benchmark.json (pending)